USB-Based Attack Research and Defense: A Technical Evaluation of Attack Vectors, Defensive Controls, and Organizational Policy

COMP 490 Independent Study Technical Report

Prepared by: Phillip Schlomann

Project Advisor: Moez Ben-Azzouz

Institution: Wittenberg University

Course: Independent Study in Cybersecurity (COMP 490)

Semester: Spring 2026

Submission Date: 04/24/2026

Executive Summary

The USB port of a computer is frequently used in the current day. From keyboard cords to wireless dongles for webcams to USB drives used to update SCADA systems. USB technology is known to be convenient and quick. These come with flaws, ones that can be exploited to cause major damage. Examples of the damage that could be caused can be found in Stuxnet and Raspberry Robin. These USB attacks can be split into five major categories: HID emulation, firmware reprogramming, network attacks, malware delivery, and power attacks. Using Kali Linux and Metasploitable3 Windows Server 2008 R2 virtual machines hosted using Oracle VirtualBox, potential USB attacks were developed for research. These attacks were primarily HID emulation with some setup as malware delivery. The payload delivery methods were with an Onn USB drive, a Digispark ATtiny85, or a Raspberry Pi Pico. The Digispark Attiny85 used Arduino IDE to write payloads, while the Raspberry Pi Pico was configured with pico-ducky to accept DuckyScript payloads written in Notepad. Among the created payloads were 4 simple keystroke injection attacks, 3 credential theft attacks, 3 backdoor persistence attacks, and 1 multi-stage attack that has a time delay and obfuscation. For defense testing, USBGuard, udev rules, Group Policy, and BIOS-level USB disabling were all tested as software solutions. USB data blockers were tested as the primary hardware solution. Endpoint detection and response, epoxy casings, and USB port locks were all researched, but were unable to be tested. This research was compiled into a comprehensive security policy, outlining the necessity of a defense-in-depth strategy and strong definitions for responsibilities and appropriate use. This policy was accompanied by an implementation guide to walk an IT department through setting up recommended USB security measures. An awareness quiz, user reference card, and awareness fliers were also created. A key finding from this research is the frequency at which the Digispark ATtiny85 bypassed security measures, as only the USB data blocker successfully blocked it. This research also emphasized the danger USB attacks pose, since many of them were viable without elevated privileges and caused great damage in a matter of seconds.

Introduction

Background and Motivation

USB devices and media are everywhere. Keyboards and mouses are used for almost every computer. These devices are usually connected through USB. USB flash drives are a ubiquitous method of external storage. They are frequently used to transfer small amounts of data, or store data when the computer can’t handle the file sizes. It is this frequency of use that served as the motivation for this research. If the USB port allows for easy connection to external devices of so many different functions, would it also allow for connection to external devices with malicious functions? If so, are there protections that would mitigate these malicious functions? To answer these questions, the background of the Universal Serial Bus is a necessary place to start.

Universal Serial Bus was made with three major motivations: Connect to PCs and phones, make PCs more accessible to use, and establish better port flexibility for peripheral devices. USB ports were unified, had great flexibility, helped lower costs for peripheral devices, and had convenient plug-and-play functionality. The idea was to have a peripheral device that the user could simply plug into a PC and start using. The flexibility was achieved by USB devices being assigned classes based on what functionality they have[1].

One of these classes is Human Interface Devices, or HID. The functionality intended is to provide humans with devices to interact with a computer. The primary goals of HID are to be compact and save data space, allow software to skip unknown information, extensible and robust, and be self-descriptive to allow for generic software[2]. However, all of these goals to be convenient for users came with some vulnerabilities. USB devices and media have autorun features to facilitate ease of use, which is easily exploitable.[3] When a USB device is designed for malicious activity, that activity will immediately execute due to the autorun features. This opens up the USB port for all types of attacks, from simple HID emulation to opening up a network with a new point of entry.

Research Question

What types of USB attacks are there?

How would one protect themselves against USB attacks?

Scope and Limitations

The research scope must be defined, as well as any potential limitations. These attacks will only be tested against virtual machines hosted on a personally owned computer. These VMs were entirely isolated from a network to prevent any hindrance or spread to other machines. The VMs were also limited in OS, as they were Kali Linux and Windows Server 2008 R2. These virtual machines only represent a small portion of the variations in their respective operating systems, with Windows Server 2008 R2 even being a legacy OS. However, the techniques used rely on

[1] USB-IF, "Universal Serial Bus Specification, Revision 2.0", Apr. 27, 2000. [Online]. Available: https://www.usb.org/document-library/usb-20-specification

[2] USB Implementers' Forum, "Device Class Definition for Human Interface Devices (HID)", 27 July 2001. [Online]. Available: https://www.usb.org/hid

[3]. Antonio dos Santos and C0d3Cr4zy , "Replication Through Removable Media, Technique T1091 - Enterprise | MITRE ATT&CK®," MITRE ATT&CK, May 31, 2017. Last Modified 24 October 2025. https://attack.mitre.org/techniques/T1091/

terminal and PowerShell commands, so should still be applicable to other, similar operating systems.

This research will also mostly focus on developing HID emulation attacks, with some malware delivery also tested. There are more methods of attack using USB devices, such as power attacks, but those will not be tested for safety reasons or limitations imposed by the methods of testing. Endpoint Detection and Response solutions like CrowdStrike Falcon were researched, though were unable to be tested since the VMs were disconnected from any network, including the internet. There were complications in being able to obtain a trial for only use in this project. It is also worth noting that the Lenovo IdeaPad1 does not have the strongest hardware installed, so many of the payload deliveries were slowed down to compensate. When these attacks are executed, keystrokes can be lost during load times if the machine does not run fast enough.

Most USB attacks are fully executed in seconds.

Literature Review

USB Attack History

The history of USB attacks is important to understand in order to see a clearer picture of USB security. One standout place to start is the 2014 BlackHat Conference. It was at this conference that Karsten Nohl, Sascha Krißler, and Jakob Lell from Security Research Labs delivered the talk "Bad USB – On accessories that turn evil". This talk outlined how USB devices can be reprogrammed to act in malicious ways, with the warning that there is no universal defense solution. These ways include keystroke injection, which can damage even without software vulnerabilities, and diversion of DNS traffic.[4] Though, these attacks weren't always this way. In prior days, buffer overflow was often the primary method of USB attack, allowing for privilege escalation. HID emulation and network attacks are far more common in the current day.[5] USB attacks of the current day are often specifically targeted as well. The groups DarkHotel and RevengeHotels used HID emulation to target the hotel industry, for instance.[6]

[4] K. Nohl, S. Krißler, & S. Lell, "BadUSB — On accessories that turn evil", Aug. 6-7, 2014. [Online]. Available: https://radetskiy.wordpress.com/wp-content/uploads/2014/08/srlabs-badusb-blackhat-v1.pdf
[5] L. Ngyuen, "The Danger of a USB Device and Keystroke Injection Attack," OPSWAT, May 13, 2024. https://www.opswat.com/blog/the-danger-of-a-usb-device-and-keystroke-injection-attack
[6] B. Ben Atair, "Bad USB Case Study," Sepio, Aug. 3, 2020. https://sepiocyber.com/resources/case-studies/badusb-attack/

Another example is perhaps one of the most notable USB attacks; Stuxnet. In January of 2010, several computers in an Iranian uranium enrichment plant crashed and rebooted. May of the same year saw even more computers crash and reboot. These crashes damaged components of the enrichment system, as well as the quality of the uranium itself. Since these systems were all air-gapped, the attack was designed to infect companies connected to the site, then spread from a computer to USB, where it can be spread to the enrichment systems from.[7] Another notable USB attack was Raspberry Robin. This was a worm that spread through USB, hiding itself in a normal-seeming CMD.exe file that downloaded the worm as a DLL when run.[8]

Attack Taxonomy

Throughout the preliminary research, five categories to assign USB attacks stood out: HID emulation, malware delivery, network attacks, firmware reprogramming, and power attacks.

HID emulation focuses on the injection of keyboard strokes from a payload on the USB device. Due to how HID works for USB-connected input devices, the payload will automatically start executing on connection, often bypassing security measures and not requiring user input.[5]

Malware delivery involves the use of a normal USB drive to deliver malware payloads through an otherwise normal USB storage drive. This payload is often stored in ways that obfuscate it, such as invisible execution when something else is clicked on or being hidden by renaming to something innocuous like “systemupdate”. One difference of malware delivery is that it does often require user input to function, hence the need to hide the payload as something harmless and/or desirable for a user.[8]

Network attacks use USB network cards or ethernet adapters to steal information from traffic generated by a computer. This takes advantage of the functionality of USB protocol, which will automatically enumerate and install a device upon connection. Network cards can be used to automatically connect to a listening machine that is acting as a DHCP server, which will allow for access to DNS and WPAD. This attack is especially dangerous since it still works the same

[7] K. Zetter, “An Unprecedented Look at Stuxnet, the World’s First Digital Weapon,” Wired, Nov. 03, 2014. https://www.wired.com/2014/11/countdown-to-zero-day-stuxnet/

[8] sangithaa@checkpoint.com, “Raspberry Robin Malware: A Worm Weaponizing USBs,” Check Point Software, Jul. 09, 2025. https://www.checkpoint.com/cyber-hub/threat-prevention/what-is-malware/raspberry-robin-malware/

[5] L. Ngyuen, “The Danger of a USB Device and Keystroke Injection Attack,” OPSWAT, May 13, 2024. https://www.opswat.com/blog/the-danger-of-a-usb-device-and-keystroke-injection-attack

on locked computers. Computers don't need to be unlocked to generate web traffic, so in only 13 seconds, a malicious USB network card could steal credential hashes from a locked computer to crack offline or use in relay attacks.[9]

Firmware reprogramming attacks see the firmware of the USB device being reprogrammed to act in a malicious way. Oftentimes these attacks will use the same techniques for delivery of malicious payloads as either a network attack or HID emulation. However, the firmware of a USB device must be known in order for it to be reprogrammed. One of the milder dangers of this attack is that it can exploit the recognizability of a USB device, using the familiarity to avoid suspicion that it has been reprogrammed. The primary complication that comes with firmware reprogramming is that most security solutions don't access the firmware of a connected USB device. Because of this, firmware reprogramming will completely dodge these solutions. What makes this even more dangerous is the methods of delivery being ones that function without user input or knowledge. Despite "BadUSB" being used as a catch-all term for USB-based attacks in some places, it is specifically a firmware reprogramming attack.[4]

Power attacks, unlike the others in this taxonomy, are pure hardware damage rather than any kind of software damage and/or data exfiltration. These attacks are the most uncommon, though they are highly damaging. These USB devices act as batteries that charge through the USB port of a computer when connected. Once they are fully charged, they will be discharged back into the port.[10] This will be very likely to damage the port, with possible damages reaching all the way up to burning out the entire computer. These attacks are also incredibly complicated to defend against, require devices like diodes, HSPs, and gas discharge tubes specially made for the function. They will even burn through common hardware solutions like USB data blockers.[11] Due to the danger of these attacks and damage associated with them, alongside the complication of defending against them, they will be kept out of the focus of the report and creation of them was not researched.

---

[9] L. Constantin, "A USB device is all it takes to steal credentials from locked PCs," Network World, Sep. 08, 2016. https://www.networkworld.com/article/955078/a-usb-device-is-all-it-takes-to-steal-credentials-from-locked-pcs-2.html

[10] D. Bolton, "Russian computer researcher creates the 'USB Killer' thumb drive that will fry your computer in seconds | The Independent," The Independent, Oct. 16, 2015. https://www.the-independent.com/tech/russian-computer-researcher-creates-a-usb-killer-thumb-drive-that-will-fry-your-computer-in-seconds-a6696511.html

[4] K. Nohl, S. Krißler, & S. Lell, "BadUSB — On accessories that turn evil", Aug. 6-7, 2014. [Online]. Available: https://radetskiy.wordpress.com/wp-content/uploads/2014/08/srlabs-badusb-blackhat-v1.pdf

[11] Bourns, "Bourns | USB Kill Threats," Bourns.com, 2026. https://bourns.com/resources/technical-library/library-documents/circuit-protection-technical-library/protect-mobile-devices-from-usb-kill-threats

Current Defense State of Practice

There are many security solutions that are commonplace when it comes to securing against USB attacks. Software solutions like USBGuard and Group Policy rules are common starting points that depend on operating system. Hardware solutions include USB data blockers and port locks. Epoxy can also be used for security, by either completely sealing unused ports or sealing the components of a USB device against tampering. More advanced solutions like behavioral-based endpoint detection and response systems also exist to prevent USB attacks. The current suggested route to take for USB security is employing a defense-in-depth strategy.[12] Many of these solutions should be implemented all at the same time. Possibly even implement all of these solutions, should the environment allow. With many different security methods in place, it makes it less likely for an attack to get through defenses since it would have to bypass all of them, rather than focusing on a weakness in just one of them.

Methodology

Lab Environment Setup

The lab was split between two different virtual machines; one is a Kali Linux VM and the other is a Metasploitable3 Windows Server 2008 R2 VM. The payloads were written on a Lenovo IdeaPad1 running Windows 11. Oracle VirtualBox was used as the hypervisor for these VMs, with the Lenovo IdeaPad1 serving as the host machine. Snapshots of these VMs would be used in order to ensure reproducibility for repeated testing. The main snapshot used was one that had the drivers for the USB devices used installed, so that each attack test would be using the same driver install.

The hardware consisted of a Raspberry Pi Pico, multiple Digispark ATtiny85s, and Onn 2GB USB Drives. Jumper wires were also used to setup the Raspberry Pi Pico and Digispark ATtiny 85s, and the Raspberry Pi Pico needed a USB-A to USB-C cord to connect to the Lenovo IdeaPad1.

The payloads for the Digispark ATtiny85s were written in Arduino IDE 2.3.7. Arduino IDE was installed with the Digispark AVR Boards package from the IDE itself and the Digispark drivers

[12] AtliQSEOTeam, "Physical Port Locks vs Software Security: Which One Works Better?," Smart Keeper, Jan. 16, 2026. https://smartkeeper.us/physical-port-locks-vs-software-security-which-one-works-better/

from Digistump's GitHub. The board type of these ATtiny85s was "Digispark (default – 16.5Mhz)".

The payloads for Raspberry Pi Pico were written in Notepad, following syntax for DuckyScript. The Raspberry Pi Pico had the code from the pico-ducky GitHub repository installed on it. This project allows for a Raspberry Pi Pico to function similarly to a Rubber Ducky, which is a USB device that HID emulation payloads are stored on to immediately execute when connected. The most important aspect of pico-ducky is that the payloads that it uses are written in DuckyScript, the language of Rubber Ducky payloads. This installation followed a few steps. The Raspberry Pi Pico was connected to the Lenovo IdeaPad1 while holding BOOTSEL to enter setup mode. At this point, the "adafruit-circuitpython-raspberry_pi_pico-en_US-10.0.3.uf2" file was copied to the root of the device, which caused it to reboot and reconnect as "CIRCUITPY". Once that was done, the lib folder and all files with the .py extension from the pico-ducky code were copied to the root of the Raspberry Pi Pico. The device was then ready to have payload.dd files copied to it after connecting it with a jumper wire between pins 1 and 3 for it to connect in setup mode.[13]

Attack Development Process

The first few attack payloads created were focused on much simpler tasks such as opening a terminal window in Linux or a PowerShell window in Windows. Because of this, the early attacks allowed for learning of the Arduino IDE and DuckyScript languages without complex logic or commands to use. These early payloads served as the basis for the rest of the attacks, with the knowledge of the language being used as well as many of the same techniques. Knowledge from previous experience with these machines also served to help facilitate the functionality of these attacks. The work of others was often referenced to see how the languages worked. When necessary, techniques for later attacks were also researched. These attacks would often be tested outside of their recorded testing count in order to ensure they would function.

Defense Testing Protocol

Most tests focused on the functionality of the developed attacks without any defense measures in place. For these tests, a snapshot of the VM with all of the set-up complete. The snapshot would be loaded and then the payload delivered, noting if the attack was successful or not. Each attack was tested a total of 7 times. When it came to testing the defense measures, a snapshot was taken once that measure was set-up, then a device would be tested. Each device was tested on separate

[13] dbisu, "pico-ducky," GitHub.com, Last Updated Jan. 10, 2026. https://github.com/dbisu/pico-ducky

loads of the snapshot, and each device was tested at least 3 times across separate loads of the snapshot.

Ethical Considerations

Given that this research will include the development and testing of genuine malicious payloads for USB attacks, there are some notable ethical considerations. These payloads were created for research purposes only and were only used on machines I had full permissions to test on; personally-owned virtual machines on a personally-owned host machine. These payloads and the knowledge gained from them are not to be used for any malicious purposes in the future and sharing of them will be for educational purposes only.

Attack Development Results

Keystroke Injection Attacks

There was a total of 4 payloads created to make simple keystroke injection attacks. These payloads were OpenTerminalLinux, OpenPowershellWindows, OpenReverseShellLinux, and DownloadExecuteScriptLinux. All of the payloads for these attacks were made in Arduino IDE for the Digispark ATtiny85.

OpenTerminalLinux uses a keyboard shortcut to open a Terminal window, which forms the basis of many attack attempts in Linux.

OpenPowershellWindows uses a keyboard shortcut to open the Windows Run Dialogue, then types in PowerShell followed by pressing Enter to launch a PowerShell window. Many further attacks in Windows use this script as a basis.

To set up for OpenReverseShellLinux, a netcat listener was opened in a separate Terminal window in Linux. This is where the snapshot was taken for testing. When the Digispark ATtiny85 is plugged in, the payload opens a Terminal window, then types in a command to establish a reverse bind shell with the listener window, pressing Enter afterward to execute the command.

To set-up for DownloadExecuteScriptLinux, a sample bash script was written using vim, then a Python3 HTTP server was opened in a separate window to serve the file. This is where the

snapshot was taken for testing. When the Digispark Attiny85 is plugged in, the payload opens a Terminal window, then types several commands, each followed by a press of Enter to execute them. First, it changes directory to /tmp in an attempt to hide the download of the script. Next, a wget command is used to download the script to the current directory. After downloading the script, the permissions are changed to allow for the script to be executable. Finally, the script is called and executed.

Credential Theft Payloads

There was a total of 3 payloads created to act as credential theft attacks. These payloads were ShadowPOSTExfiltration, WindowsCredentialManager, and MimikatzExfiltration. ShadowPOSTExfiltration is a Linux payload while WindowsCredentialManager and MimikatzExfiltration are both Windows payloads. ShadowPOSTExfilitration was developed to be a DuckyScript payload, WindowsCredentialManager was delivered with the Digispark ATtiny85, and MimikatzExfiltration used the USB drive to mimic a malware delivery drop attack.

To set up for ShadowPOSTExfiltration, a while true; do loop with netcat was set up as an HTTP listening server to receive the shadow file. This is where the snapshot was taken for testing. After this, the Raspberry Pi Pico is connected and the payload is executed. The payload will use a curl command to send the contents of the /etc/shadow file to the listener through HTTP POST. Then the payload will clear the terminal history and close the terminal window in an attempt to hide itself. Note that this attack requires sudo privileges, since it reads the /etc/shadow file.

To set up for WindowsCredentialManager, a server PowerShell script was written and run to start an HTTP server to receive a POST request. This is where the snapshot was taken for testing. When the Digispark ATtiny85 is plugged in, it copies the contents of the Credential Manager to a file named temp.tmp in an attempt to hide the file. That file is then sent to the server through a POST request using the Invoke-WebRequest cmdlet.

To set up for MimikatzExfiltration, a server PowerShell script was written and run to start an HTTP listener to receive a POST request. This is where the snapshot was taken for testing. Tthe Onn USB drive had an install.bat, install.ps1, and mimikatz.exe with its DLLs placed on it for use in the attack. install.ps1 and the mimikatz files are hidden inside a x64 folder while install.bat is in the root of the drive. When install.bat is run, it calls the install.ps1 script while bypassing execution policy and making a terminal opening unnecessary. install.ps1 uses a mimikatz command to read the passwords of the machine, bypassing any necessary privileges.

This is then redirected to a passwords.txt file on the USB, which is then sent to the listener as a POST request using the Invoke-WebRequest cmdlet. Note that this attack could also be done by downloading mimikatz from the internet, should the target machine have internet access.

Persistence Mechanisms

There was a total of 3 payloads created to demonstrate how persistence can be established in USB attacks. These payloads were CronJobsBackdoor, RegistryBackdoor, and ScheduledTasksBackdoor. CronJobsBackdoor and ScheduledTasksBackdoor were created for Linux, while RegistryBackdoor was created for Windows. RegistryBackdoor and ScheduledTasksBackdoor were delivered with a Digispark ATtiny85, while CronJobsBackdoor was delivered with the Raspberry Pi Pico with a payload written in DuckyScript.

To set up for CronJobsBackdoor, a netcat listener was opened. This is where the snapshot for testing was taken. Once the Raspberry Pi Pico is connected, the payload creates a system_update.sh script in /tmp that establishes a reverse bind shell with the listener, then changes the permissions of that script to make it executable and make a cron job that runs the system_update.sh script every minute. Finally, the payload ends by closing the terminal window it opened.

To set up for RegistryBackdoor, netcat.exe was transferred to the Windows machine via a clean Onn USB drive and renamed to winconfig.exe. Netcat is necessary for this attack, so the executable was moved to the machine. Typically, this would be achieved by having Netcat downloaded from the internet as part of the attack, though this is not possible on the specific VM used. This is where the snapshot for testing was taken. When the Digispark ATtiny85 is plugged in, there are a few things it does. The first thing it does is to open a terminal window and create a temp.ps1 script that opens a netcat bind shell listener. After this, it creates a run.bat file that runs the tmp.ps1 script, bypassing execution policy. Then, a Registry Run Key is created that runs the run.bat file when the current user logs on. Finally, it forces the current user to log off. Upon logging back on, a terminal window will open with no visible commands. This is the reverse bind shell listener waiting for connection. Normal antivirus solutions often don't detect Registry changes, so this payload would likely go unnoticed. A more advanced solution would be required to detect it.[14]

[14] T. Contreras, "From Registry With Love: Malware Registry Abuses", splunk.com, Jan. 19, 2023. https://www.splunk.com/en_us/blog/security/from-registry-with-love-malware-registry-abuses.html

To set up for ScheduledTasksBackdoor, is also needed, so it shared a snapshot with RegistryBackdoor. When the DigisparkATtiny85 is plugged in, it creates a Scheduled Task to run a netcat command that opens a bind shell listener when the user logs in, then it logs the current user off. Upon logging in, the running bind shell listener will only be visible in the notifications bar. Windows Defender is capable of detecting potentially harmful Scheduled Tasks being run, so this backdoor may be detected and flagged.[15]

Obfuscation Techniques

One main payload was created to demonstrate a multi-stage attack where the primary script is downloaded by the device in an obfuscated manner with a time-delayed component to the attack. This payload was DownloadDelayObfuscated. This payload was written as a DuckyScript file for the Raspberry Pi Pico. To set up for this payload, a netcat reverse bind shell listener was started in a separate terminal window. Then, a shell script was written that waits 90 seconds, then connects to the reverse bind shell listener. This script is named .update.sh in an effort to hide it, then a Python3 HTTP server was started to serve the file. This is when the snapshot for testing was taken. After the Raspberry Pi Pico is connected, it will open a terminal window and proceed to run a series of commands encoded in Base64 that will download .update.sh, change its permissions to make it executable, and finally run it. After 90 seconds, the reverse bind shell listener will make connection.

Attack Success Rates

OpenTerminalLinux	OpenPowershellWindows	OpenReverseShellLinux	DownloadExecuteScriptLinux	ShadowPOSTExfiltration	WindowsCredentialManager	MimikatzExfiltration	CronJobsBackdoor	RegistryBackdoor	ScheduledTasksBackdoor	DownloadDelayObfuscated
7/7	7/7	1/7*	5/7	5/7	6/7	6/7	3/7	6/7	7/7	6/7

* Delays between keystrokes were not long enough, leading to premature typing of commands

Defensive Measures Evaluation

Software Defenses

[15] JohnSmith1910, “Microsoft Defender "Suspicious Scheduled Task Launched”, PSAppDeploymentToolkit, Feb. 01, 2023 https://discourse.psappdeploytoolkit.com/t/microsoft-defender-suspicious-scheduled-task-launched/4476

See Appendix A for full implementation notes

USBGuard

USBGuard was configured with the following commands: sudo apt install usbguard usbutils udisks2[16] -> service usbguard status -> sudo -i -> usbguard generate-policy > /etc/usbguard/rules.conf -> exit -> update-rc.d -f usbguard remove -> update-rc.d -s usbguard defaults. These commands start by installing USBGuard and its dependencies, then checking the status to verify installation. After this, a root user session is started to generate a policy to block devices except for ones currently connected. The root session is then closed, USBGuard has any symlinks removed, and then it is fully initialized. Once USBGuard is configured, it can be started by running the following command: service usbguard start. Stop, restart, and status are all also commands callable for USBGuard that allow for further interactions with the service.[17] Note that this solution does require sudo privileges.

udev Rules

The udev rule was implemented with the following commands: cd /etc/udev/rules.d -> sudo su -> touch 10-usbblock.rules -> vim 10-usbblock.rules -> in usbblock.rules; #ACTION=="add", ATTR{bInterfaceClass}=="03" RUN+="/bin/sh -c 'echo 0 >/sys$DEVPATH/../authorized'" -> -s -> sed -i 's/#//' /etc/udev/rules.d/10-usbblock.rules; udevadm control --reload-rules -> exit -> exit. This series of commands will create a new usbblock rules file in the rules directory of the udev directory. After creating this file, the rule will be written in vim that will block any new USB devices or media, though continue to allow connected ones. The file is then saved and activated by reloading the udev rules. Finally, running exit twice will leave root mode and then exit out of terminal session.[18] Note that this method requires a root session to be created, so extreme caution is necessary when implementing this rule.

Group Policy

The Group Policy Object was created with settings in the following path: Computer Configuration -> Policies -> Administrative Templates -> System -> Removable Storage Access

---

[16] Vivek Gite, "How to protect Linux against rogue USB devices using USBGuard - nixCraft," nixCraft, Jul. 11, 2022. https://www.cyberciti.biz/security/how-to-protect-linux-against-rogue-usb-devices-using-usbguard/

[17] NI, "USBGuard Installation and Configuration — nilrt-docs 1.0 documentation," Ni.com, 2026. https://nilrt-docs.ni.com/usbguard/usbguard.html

[18] Albert and user10008, "How to prevent BadUSB attacks on Linux desktop?," Information Security Stack Exchange, Aug. 03, 2014. Last edited May. 18, 2017. https://security.stackexchange.com/questions/64524/how-to-prevent-badusb-attacks-on-linux-desktop

of GPO Management. Once in Removable Storage Access of GPO Management, the following changes are made: Removable Disks: Deny Execute Access is enabled, Removable Disks: Deny Write Access is enabled, and Removable Disks: Deny Read Access is enabled. Prevent installation of removable devices is also enabled from the following path: Computer Configuration -> Policies -> Administrative Templates -> System -> Device Installation -> Device Installation Restrictions. This Group Policy Object was then applied to the computer. These Group Policy rules will remove the read, write, and execute privileges of removable media and files on removable media, as well as prevent any removable devices from installing onto the machine. These methods will make it difficult for a USB drive to deliver malware or any HID emulation attacks to execute. One thing to note about these restrictions is that it will not restrict any currently detected USB devices.[19]

BIOS-Level USB Disabling

As these machines were VMs using Oracle VirtualBox as their hypervisor, they didn't have traditional BIOS. To mimic disabling of USB media and devices from a BIOS-level, the USB Controller was disabled in the VirtualBox settings for the VM being tested.

EDR Solutions

Microsoft Defender ADR is only for Windows OS, allows for whitelisting and blacklisting of external storage and devices, provides highly granular control, offers reports and advanced threat hunting, and can be seamlessly integrated with BitLocker, Intune, and Group Policy.[20] ADR also provides real-time scanning and protection from execution and direct memory access until the device or media is fully verified.[21]

SentinelOne is designed for both macOS and Windows OS and allows for high granularity and flexibility, being controlled by Device Control rules that are device group-level, site-level, or enterprise-level. SentinelOne allows for restriction to read-only, read-write-only, or fully block

---

[19] Anonymous, "Create A GPO to disable USB Storage Devices - Microsoft Q&A," Microsoft.com, Dec. 08, 2023. https://learn.microsoft.com/en-us/answers/questions/2198881/create-a-gpo-to-disable-usb-storage-devices

[20] siosulli, "Device control in Microsoft Defender for Endpoint - Microsoft Defender for Endpoint," learn.microsoft.com, May 15, 2024. https://learn.microsoft.com/en-us/defender-endpoint/device-control-overview

[21] Microsoft Threat Intelligence, "Windows Defender ATP has protections for USB and removable devices | Microsoft Security Blog," Microsoft Security Blog, Dec. 19, 2018. https://www.microsoft.com/en-us/security/blog/2018/12/19/windows-defender-atp-has-protections-for-usb-and-removable-devices/

USB devices and media, as well as Bluetooth devices. There is also a Device Log kept that stores all blocked devices.[22]

CrowdStrike Falcon works on all three major operating systems. It uses deep learning to analyze source code and provides a host of information about connected USB devices or media. It is even able to track down renamed or hidden activity and scan and enumerate files in ZIP archives.[23]

Hardware Defenses

USB data blockers were the primary hardware defense used. This was done by putting the data blocker into the port of the host machine, then loading up the virtual machine that was being tested from a snapshot that had drivers installed but no security in place. Each device used on the VM being tested would then be connected to the data blocker, recording whether or not the device connected. Each VM had the devices used on it test separately. Data blockers do have some weaknesses worth bringing up. While inexpensive and convenient for a single user, outfitting an entire enterprise environment with data blockers for all necessary ports will end up costing a high amount. There would also be a serious logistical challenge to properly account for and keep track of every individual data blocker. Data blockers also would fail to prevent a power attack. It is worth noting that situations in an enterprise environment where only power is needed through a USB port would also be uncommon. For these reasons, data blockers likely wouldn't be practical in an enterprise environment. However, on an individual level, they are an invaluable tool for preventing attacks or transfer of malware when it comes to charging mobile devices through a USB port.

While unable to be tested, other hardware defense methods were researched. One of these methods was the use of epoxy for USB security. One way of using epoxy is to permanently block unnecessary ports on machines. This method entails filling the port with epoxy to remove that port's functionality permanently. This is an irreversible measure and comes with some serious side effects, so should be carefully considered. Filling the port with epoxy will void the warranty and can damage the hardware if the resin isn't allowed to fully cure before the machine is turned on.[24] The other method of using epoxy is to seal the components of a USB device or media with

[22] E. Erlich, "Feature Spotlight - Enhanced USB & Bluetooth Device Control," SentinelOne, Jul. 30, 2020. https://www.sentinelone.com/blog/feature-spotlight-enhanced-usb-bluetooth-device-control/
[23] D. Fernandez, "Mitigating USB Data Exfiltration With Falcon Device Control | CrowdStrike," Crowdstrike.com, 2026. https://www.crowdstrike.com/en-us/blog/mitigating-usb-data-exfiltration-with-new-capabilities-in-falcon-device-control/
[24] P. Goldstein, "Disabling USB ports: 4 ways to prevent data leaks via USB devices," Technology Solutions That Drive Government, Jul. 07, 2017. https://fedtechmagazine.com/article/2017/07/4-ways-prevent-leaks-usb-devices

epoxy. This will secure the components against any form of tampering with a tamper-evident seal. The components will still be susceptible to reading of electromagnetic emissions with TEMPEST but will be protected from attackers researching susceptibility in a device's components.[25]

Another researched hardware defense was USB port locks. These are plugs designed to connect to a USB port and lock in place, blocking it from use but allowing for removal later with a corresponding key. These plugs allow for customization as well, with the potential for color coding based on key and unique keys being made for an institution.[26]

Policy and Awareness

Alongside these defense measures, a sample comprehensive USB security policy, awareness fliers, and an awareness quiz were created. The policy reflects all of the knowledge gained from research and testing, approaching USB security with many solutions to create layered security. The recommended controls were to use USB port locks with corresponding company-specific keys with a recommendation to not reuse keys, use locks for always-connected devices like keyboard and mouse, enable an EDR solution on all infrastructure that allows, enable an antivirus solution on all infrastructure that allows, enable Group Policy on Windows machines, enable USBGuard on Linux machines, and store company-used USB devices and media in a highly secure closet, requiring documentation for all that is done with the device. The policy outlined definitions that acted to help strengthen its enforceability. The people the policy applies to as well as the situations and methods they are permitted to use USB devices and media are defined, as are the responsibilities of some roles in the company and potential consequences for those found to be in violation of the policy.

Alongside this policy, an implementation guide was created to fully walkthrough how to set-up the security measures outlined by the policy, with some additional requirements also present. Employees would also benefit from extra reminders, so two fliers referencing the security policy in a short and simple manner were made to hang around and office.

A one-page condensing of the most important end user-facing components of the USB security policy was created to act as a quick reference. Beyond this, a user awareness quiz was also

[25] A. Drozhzhin, "Is your encrypted USB drive secure?", Kaspersky, Aug. 10, 2017. https://www.kaspersky.com/blog/encrypted-usb-drives-audit/17948/
[26] E. Greenberg, "USB Blockers: Physical Security Solutions for Critical Port Protection," Sasa Software, Apr. 24, 2025. https://www.sasa-software.com/learning/what-is-a-usb-blocker/

created to test the knowledge of USB security. This quiz is a simple 10-question multiple-choice quiz with scenarios outlined for each question. For the first 5 questions, the scenarios are wholly independent. However, the second 5 questions have an overarching security policy reminder to guide the user. This split is to potentially test pure instinct vs. ability to follow policy in the context of USB security.

Defense Effectiveness Analysis

	USB Drive	Digispark	DuckyScript Pico
udev Rules	Passed	Passed	Blocked
USBGuard	Blocked	Passed	Blocked
Group Policy	Blocked	Passed	
Data Blocker (Windows)	Blocked	Blocked	
Data Blocker (Linux)	Blocked	Blocked	Blocked
USB Controller Disabled (Windows)	Blocked	Passed	
USB Controller Disabled (Linux)	Blocked	Passed	Blocked

Organizational Recommendations

Layered Defense Strategy

When defending workstations from USB-based attacks, a layered defense strategy is necessary. Using only one implementation, no matter how good, will not suffice for defense against these attacks. When only one implementation is present, it only takes a vulnerability in that one implementation to bring the whole network down and/or have a massive data breach.

On the hardware side, USB port locks and epoxy should be used. USB ports that aren't currently in use should have locks installed, while using epoxy to both fill ports that aren't to be used at all and secure the components of approved USB devices and media against tampering. USB devices that are to be connected to the workstation at all times such as keyboards or mice should have their cord or dongle locked to the workstation as well.

Software Solutions

When it comes to software security, USBGuard/Group Policy, antivirus, and EDR combined should prove sufficient for most attacks. USBGuard and Group Policy will depend on the OS of the machine, as will the specific antivirus. The EDR solution may also vary based on OS, though some like CrowdStrike Falcon are usable on every OS, which can prove quite efficient for multi-

OS environments. While specific solutions may vary, they all should have similar functionality. These solutions all provide different defenses as well. The antivirus will help detect any leftover malware delivered by the connection of a USB, though does lack when it comes to preventive defense.

The implementation of USBGuard on Linux systems or Group Policy rules on Windows systems will help restrict the USB devices or media that are allowed to connect to the system but won't restrict the behavior of devices or media that do connect.

The EDR solution will help with active security, being able to stop threats when they are found and scan USB devices and media before they fully connect to the system. The behavioral-based nature of EDR solutions help detecting threats.

Signature-based detection like antivirus implementations will only be able to detect threats based on the code itself by referencing a table of known threats. This makes the solution much quicker and less likely to flag a false positive but also requires the threat to be previously known and the table must be frequently updated. Behavioral-based will detect on the actions a potential threat is taking. The solution will see what processes are running on the system and compare them to a baseline, flagging them if they deviate. This provides much more robust capability for detection, but does come with the downside of false positives, especially while initializing the baselines.[27]

All of these security solutions are important, though perhaps the strongest and most important solutions for USB security are proper user awareness and strong policies. Employees must be trained on USB security practices as well as security practices involving actors who may try to leverage USB devices or media. Having quick references and awareness fliers wouldn't hurt to have around as reminders. Training employees is one of the most important security measures as having all of the security solutions, hardware and software, won't matter if your employees don't know how to use them. Policies act as the outline for what security solutions to implement. Policies also outline proper behavior from employees and how to address violations of the policy. Interconnected policies must also be strong, as they will define processes like disposal of USB media or devices and procedure for testing new USB media or devices. Without strong policy, security measures can't be implemented as there is no way to enforce their use.

[27] SentinelOne, "Signature-Based Vs. Behavioral AI Detection: Full Comparison," Sentinelone.com, Oct. 28, 2025. https://www.sentinelone.com/cybersecurity-101/cybersecurity/signature-based-vs-behavioral-ai-detection/

Policy Framework

As part of this research, a comprehensive USB security policy was written reflecting on the knowledge gained. The key to a strong policy is explaining everything explicitly, clearly, and thoroughly. Vague definitions will only serve to make the policy unenforceable. The first thing that should be done is to outline the scope and purpose of the policy, making it clear what scenarios the policy will apply in and what it aims to do. Then, outline the responsibilities of primary roles in the organization to ensure that their jobs are enforceable. Afterward, the required security measures should be outlined. These measures should reflect the current industry best practices of approaching with a defense-in-depth strategy, employing many different types of security controls to stop the same potential threats. Afterward, an incident response plan can be laid out or referenced and the process for requesting exceptions should be detailed. Once that is done, the potential punishments for violation of the policy are to be outlined. Finally, an outline for the review cycle of when the policy should be checked for needed changes.

Implementation Roadmap

Each workstation or machine in the environment will need to have the same security measures configured. Because of this, having a roadmap for the IT team to follow will be crucial in ensuring that the configuration is repeatable and consistent across all infrastructure. The research done seemed to suggest the following implementation roadmap; Antivirus -> EDR -> USBGuard/Group Policy -> Clean USB connection -> Lock cables, dongles, & ports.

Antivirus should be set-up first since they come pre-installed in some operating systems like Windows. Windows Defender is perfectly serviceable for a Windows system, only requiring some tooling to fit the specific needs of the system. Linux systems don't have as many options for antivirus software, but a basic one like ClamAV will still aid in security. It is often said that Linux systems are secure enough that an antivirus is unnecessary, but few security precautions are wholly unnecessary when it comes to protecting company data. These antivirus implementations will not act as a primary security measure, but having one to quickly scan for any potential threats is an invaluable tool to put in place. Configuring the antivirus first allows for an initial scan of the machine to check if there was any malicious interference while it was going through the supply chain.

After setting up an antivirus, including the computer into the appropriate EDR solution is to be done. Implementing EDR solution at this point will help to catch potential threats from this point on as well as introducing the new system to the EDR's baseline.

After this, the appropriate measure for the OS will be initialized; either USBGuard or Group Policy Object. USBGuard will have to be installed, which will require an internet connection. Distribution of Linux will also decide how installation is done, as some use yum and others use apt. The Group Policy Object is to apply to everyone and every device, with it acting to disable the use of anything related to removable media or devices.

Once this is done, the verified USB media or devices the workstation will need for function, such as mouse and keyboard, should be connected. Connecting them at this time will allow the IT team to familiarize themselves with exceptions processes on the workstation. This order allows for security measures to already be in place should the safe devices cause malicious behavior.

Finally, once those are connected, the cables and dongles connected to them should be locked in place, while the remaining ports have locks installed as well. These locks will limit access to the USB ports on the machine in a method that is tamper-evident.

Some infrastructure, like SCADA systems or servers, may not support the ability to run some of these measures. In this case, the utmost importance should be placed on physically protecting the infrastructure in question.

Discussion

Key Findings

Digispark ATtiny85 bypassed all security solutions except the hardware data blocker.

This research led to many findings regarding USB attacks and security. One of the most important findings is the prevalence of the Digispark ATtiny85 devices bypassing common security solutions. The frequency at which attacks failed is also worth noting, since it reveals a major issue with HID emulation attack security. Since the Digispark ATtiny85 is considered as an HID to the target machine, the enumeration for it to connect occurs immediately upon plugging it in without user intervention. There is also no strict data transfer back to the Digispark, since it only holds the payload. This means it bypasses anything that attempts to stop USB media or transfer of data to a USB device or media. When injecting keystrokes, there needs to be built-in delays long enough to allow for previous commands to finish executing, otherwise the whole payload is offset and fails to deliver successfully. These delays will likely vary on the

machine's hardware and also reduce stealth level the longer they are. However, the potential danger of USB attacks was also shown. Many of these attacks were able to be executed with user accounts that had low privileges, even including some password exfiltration. When it comes to HID emulation, even the average guest account can cause major damage. These attacks also tend to be very fast. The HID emulation and malware delivery attacks developed and demonstrated took less than a minute to do things like establishing a reverse bind shell or exfiltrate passwords. Despite potential weaknesses, the speed and flexibility of these attacks make USB security a must in an enterprise environment.

Regarding the quiz results, there was a sample size of n=11. Part 1 had an average of 87% and Part 2 had an average of 89%. While Part 2's average was higher, the results aren't far enough apart to make a definitive statement of correlation between higher scores and an outlined policy. Part 2 also had more variance in the questions that were missed. See Appendix B for the full quiz results on each question.

Limitations

As previously mentioned in the Scope and Limitations section, there is some context to consider when looking at the results of this research. All of these tests were performed on virtual machines rather than native machines. These VM images were also designed specifically for cybersecurity rather than being business-minded operating systems. This is especially true when regarding the Windows VM, as Windows Server 2008 R2 is a nearly 2-decade old operating system, predating many new security features of Windows, such as Windows Defender. The attacks were also limited to the lab set-up, which had the VMs entirely isolated. As such, network attacks were not tested and internet downloads were not incorporated into the payloads. The quiz results were limited to people taking the quiz fairly and honestly, as well as the questions being different in both parts possibly leading to more trick questions in one part than the other.

Future Research Directions

There are attack methods and defense solutions that were unable to be tested due to the scope and limitations of the research. Since the only Windows virtual machine tested was a legacy OS with many known vulnerabilities, it doesn't give a perfect example of what USB security looks like on modern Windows systems. Most importantly, Windows Server 2008 R2 pre-dates Windows Defender, so any potential security provided by that accessible and common solution was missed. Being able to test these payloads against a Windows 10 or 11 VM would be the next step.

Endpoint Detection & Response solutions were unable to be deployed since the VMs were completely isolated and trials were unable to be acquired. Setting up a proper lab environment with multiple running VMs and making an account for an EDR solution to receive a test period would address this limitation.

Since these VMs were completely isolated from any network, any potential issues exfiltrating across networks or the internet were missed. Network attacks using something like a LAN Turtle were also not able to be tested. Setting up a running network with internet capabilities for exfiltrations or downloads during the attacks would be the next step.

The machines used to test were VMs, not normal hardware machines. Proper BIOS-level USB disabling was unable to be tested for this reason. Being able to test on personally-owned devices would address this limitation.

Conclusions

The utilities implemented by Universal Serial Bus 2.0 did great work in simplifying the use of computer peripherals, but these also serve as a weak point. The autorun and enumeration techniques used to give USB media and devices their plug-and-play functionality open up for malicious USB devices to bypass security measures. For instance, USB drives can be loaded with malware that discretely infects a computer or a device that emulates the functions of a keyboard to inject malicious keystrokes. Because of this, strong security measures are necessary. Software measures like USBGuard, antivirus, and endpoint detection and response serve to detect and stop USB-based attacks paired with hardware measures like USB port locks and epoxy coatings will form a strong security posture against USB attacks. Strong security must also be paired with strong policy and user awareness. The policy must be comprehensive and work alongside intended security measures to create a cohesive strategy. The policy must also outline what is appropriate USB use, who it applies to, and what consequences they may face for violating the policy. Users must also be trained in secure USB practices. This can be done in the form of awareness quizzes, references for the policy, and short fliers to post in the office.

AI Disclosure

Google's Gemini AI tool was used as an aid for finding references and writing the HTTP POST PowerShell server. The rest of the paper, including written content, analysis, and recommendations are the author's own work.

References

USB-IF, "Universal Serial Bus Specification, Revision 2.0", Apr. 27, 2000. [Online]. Available: https://www.usb.org/document-library/usb-20-specification

USB Implementers' Forum, "Device Class Definition for Human Interface Devices (HID)", 27 July 2001. [Online]. Available: https://www.usb.org/hid

A. Ott, "HIDRAW - Raw Access to USB and Bluetooth Human Interface Devices — The Linux Kernel documentation," *Kernel.org*, 2019. https://docs.kernel.org/hid/hidraw.html

J. Antonio dos Santos and C0d3Cr4zy , "Replication Through Removable Media, Technique T1091 - Enterprise | MITRE ATT&CK®," *MITRE ATT&CK*, May 31, 2017. Last Modified 24 October 2025. https://attack.mitre.org/techniques/T1091/

No External Contributors, "Hardware Additions, Technique T1200 - Enterprise | MITRE ATT&CK®," *MITRE ATT&CK*, Apr. 18, 2018. https://attack.mitre.org/techniques/T1200/

K. Zetter, "An Unprecedented Look at Stuxnet, the World's First Digital Weapon," *Wired*, Nov. 03, 2014. https://www.wired.com/2014/11/countdown-to-zero-day-stuxnet/

K. Nohl, S. Krißler, & S. Lell, "BadUSB — On accessories that turn evil", Aug. 6-7, 2014. [Online]. Available: https://radetskiy.wordpress.com/wp-content/uploads/2014/08/srlabs-badusb-blackhat-v1.pdf

B. Ben Atair, "Bad USB Case Study," *Sepio*, Aug. 3, 2020. https://sepiocyber.com/resources/case-studies/badusb-attack/

T. Thomas, M. Piscitelli, B. Ashok Nahar, I. Baggili, "Duck Hunt: Memory Forensics of USB Attack Platforms", Aug. 14, 2021. [Online]. Available:

https://digitalcommons.newhaven.edu/cgi/viewcontent.cgi?article=1099&context=electricalcomputerengineering-facpubs

L. Ngyuen, "The Danger of a USB Device and Keystroke Injection Attack," *OPSWAT*, May 13, 2024. https://www.opswat.com/blog/the-danger-of-a-usb-device-and-keystroke-injection-attack

S. Neuner, A. G. Voyiatzis, S. Fotopoulos, C. Mulliner, E. R. Weippl, "USBlock: Blocking USB-Based Keypress Injection Attacks," Dec. 13, 2013 [Online]. Available: https://inria.hal.science/hal-01954405v1/document

A. Nason, "Why USB Attacks Are Back And How To Prevent Them | Coro Cybersecurity," *www.coro.net*, Mar. 18, 2024. https://www.coro.net/blog/why-usb-attacks-are-back-and-how-to-prevent-them

sangithaa@checkpoint.com, "Raspberry Robin Malware: A Worm Weaponizing USBs," *Check Point Software*, Jul. 09, 2025. https://www.checkpoint.com/cyber-hub/threat-prevention/what-is-malware/raspberry-robin-malware/

Sophie, "Detecting USB-based cyber attacks: A guide," *Hack The Box*, Mar. 21, 2022. https://www.hackthebox.com/blog/detecting-usb-cyber-attacks-guide

L. Constantin, "A USB device is all it takes to steal credentials from locked PCs," *Network World*, Sep. 08, 2016. https://www.networkworld.com/article/955078/a-usb-device-is-all-it-takes-to-steal-credentials-from-locked-pcs-2.html

Bourns, "Bourns | USB Kill Threats," *Bourns.com*, 2026. https://bourns.com/resources/technical-library/library-documents/circuit-protection-technical-library/protect-mobile-devices-from-usb-kill-threats

D. Bolton, "Russian computer researcher creates the 'USB Killer' thumb drive that will fry your computer in seconds | The Independent," *The Independent*, Oct. 16, 2015. https://www.the-

independent.com/tech/russian-computer-researcher-creates-a-usb-killer-thumb-drive-that-will-fry-your-computer-in-seconds-a6696511.html

S. Wise, "reverseshell-LinuxOS.txt", June 22, 2023. [Online]. Available: https://github.com/stromwise/DIGISPARK/blob/main/reverseshell-LinuxOS.txt

A. Smith, "How do I run a command from a base64 encoded string in bash?", *ServerFault.com*, Jan. 12, 2023. https://serverfault.com/questions/896695/how-do-i-run-a-command-from-a-base64-encoded-string-in-bash

Vivek Gite, "How to protect Linux against rogue USB devices using USBGuard - nixCraft," *nixCraft*, Jul. 11, 2022. https://www.cyberciti.biz/security/how-to-protect-linux-against-rogue-usb-devices-using-usbguard/

NI, "USBGuard Installation and Configuration — nilrt-docs 1.0 documentation," *Ni.com*, 2026. https://nilrt-docs.ni.com/usbguard/usbguard.html

Albert and user10008, "How to prevent BadUSB attacks on Linux desktop?," *Information Security Stack Exchange*, Aug. 03, 2014. Last edited May. 18, 2017. https://security.stackexchange.com/questions/64524/how-to-prevent-badusb-attacks-on-linux-desktop

Anonymous, "Create A GPO to disable USB Storage Devices - Microsoft Q&A," *Microsoft.com*, Dec. 08, 2023. https://learn.microsoft.com/en-us/answers/questions/2198881/create-a-gpo-to-disable-usb-storage-devices

NNMC IT Services, "Removable Media Acceptable Use Policy," *Northern New Mexico University.* [Online]. Available: nnmc.edu/_document_repository/policies/information-technology/Removable-Media-Acceptable-Use-Policy.pdf

S. Parenti, "Enterprise USB Safety: Best Practices for Secure Removable Media Management," *Sasa Software*, Apr. 22, 2025. https://www.sasa-software.com/learning/best-practices-for-usb-safety-in-enterprises/

Red Hat, "4.12. Using USBGuard | Security Guide | Red Hat Enterprise Linux | 7 | Red Hat Documentation," *Redhat.com*, 2017. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/security_guide/sec-using-usbguard

J. Iannarelli, "What Is A USB Data Blocker And Why Do You Need One? | FBI John," *FBI John*, Jun. 24, 2023. https://fbijohn.com/what-is-a-usb-data-blocker/

T. Deluca-Smith, "What is USB Group Policy?," *Endpoint Protector Blog*, Nov. 07, 2023. https://www.endpointprotector.com/blog/what-is-usb-group-policy/

AtliQSEOTeam, "Physical Port Locks vs Software Security: Which One Works Better?," *Smart Keeper*, Jan. 16, 2026. https://smartkeeper.us/physical-port-locks-vs-software-security-which-one-works-better/

D. Strickland, "USB Drive Security Best Practices You Need to Implement Now," *CurrentWare*, Jun. 14, 2024. https://www.currentware.com/blog/usb-drive-security-best-practices/

E. Greenberg, "Implementing USB Port Lockdown: Effective Policies and Procedures for Enterprise Security," *Sasa Software*, Apr. 23, 2025. https://www.sasa-software.com/learning/how-to-implement-usb-port-lockdown-policies/

AMPCOM, "Physical USB Port Locks vs. Software Blocking – How Should Enterprises Secure Their USB Ports?," *AMPCOM*, Aug. 08, 2025. https://www.ampcom.com/blogs/industry-insights/physical-usb-port-locks-vs-software-blocking-how-should-enterprises-secure-their-usb-ports

E. Greenberg, "USB Blockers: Physical Security Solutions for Critical Port Protection," *Sasa Software*, Apr. 24, 2025. https://www.sasa-software.com/learning/what-is-a-usb-blocker/

P. Goldstein, "Disabling USB ports: 4 ways to prevent data leaks via USB devices," *Technology Solutions That Drive Government*, Jul. 07, 2017. https://fedtechmagazine.com/article/2017/07/4-ways-prevent-leaks-usb-devices

A. Drozhzhin, "Is your encrypted USB drive secure?", *Kaspersky*, Aug. 10, 2017. https://www.kaspersky.com/blog/encrypted-usb-drives-audit/17948/

siosulli, "Device control in Microsoft Defender for Endpoint - Microsoft Defender for Endpoint," *learn.microsoft.com*, May 15, 2024. https://learn.microsoft.com/en-us/defender-endpoint/device-control-overview

Microsoft Threat Intelligence, "Windows Defender ATP has protections for USB and removable devices | Microsoft Security Blog," *Microsoft Security Blog*, Dec. 19, 2018. https://www.microsoft.com/en-us/security/blog/2018/12/19/windows-defender-atp-has-protections-for-usb-and-removable-devices/

E. Erlich, "Feature Spotlight - Enhanced USB & Bluetooth Device Control," *SentinelOne*, Jul. 30, 2020. https://www.sentinelone.com/blog/feature-spotlight-enhanced-usb-bluetooth-device-control/

D. Fernandez, "Mitigating USB Data Exfiltration With Falcon Device Control | CrowdStrike," *Crowdstrike.com*, 2026. https://www.crowdstrike.com/en-us/blog/mitigating-usb-data-exfiltration-with-new-capabilities-in-falcon-device-control/

Inventive HQ, "CrowdStrike Device Control Setup: Block USB, Find Device IDs & Configure Policies," *Inventivehq.com*, 2026. https://inventivehq.com/knowledge-base/crowdstrike/how-to-setup-crowdstrike-device-control

SentinelOne, "Signature-Based Vs. Behavioral AI Detection: Full Comparison," *Sentinelone.com*, Oct. 28, 2025. https://www.sentinelone.com/cybersecurity-101/cybersecurity/signature-based-vs-behavioral-ai-detection/

dbisu, "pico-ducky," *GitHub.com*, Last Updated Jan. 10, 2026. https://github.com/dbisu/pico-ducky

S. Wilson, "A simple Python HTTP server for your sysadmin toolbox", *Red Hat Blog*, May 27, 2020. https://www.redhat.com/en/blog/simple-http-server

C. Connell, "HTTP POST from PowerShell", *Medium*, Jan. 10, 2022. https://medium.com/@chuck.connell.3/http-post-from-powershell-45a36581572

user736893, "Powershell script listen for http on public IP address", Stack Overflow, Dec. 20, 2022. https://stackoverflow.com/questions/74865579/powershell-script-listen-for-http-on-public-ip-address#:~:text=$httpListener%20=%20New%2DObject%20System.Net.HttpListener%20$httpListener.Prefixes.Add(%22http://*:5000/%22)%20$httpListener.Start()%20write%2Dhost%20%22Listening&text=%22%20do%20%7B%20$context%20=%20$httpListener.GetContext()%20$context.Response.StatusCode,Write%2DHost%20%22.%22%20%2DNoNewLine%20%7D%20until%20(%5BSystem.Console%5D::KeyAvailable)%20$httpListener.Close()

The Hacker Recipes, "Windows Credential Manager", *The Hacker Recipes*, Last updated Apr. 07, 2026. https://www.thehacker.recipes/ad/movement/credentials/dumping/windows-credential-manager

Rich, "Mimikatz Cheatsheet", *Medium*, Aug. 25, 2022. https://happycamper84.medium.com/mimikatz-cheatsheet-ad2b88059b4

S. Knight & M. Canul, "Ducky Demo", *University of Hawai'i Maui*, Last updated May 6, 2016. https://uhmc.github.io/Ducky_Demo/

refabr1k, “Ncat Persistent Backdoor”, *refabr1k’s Pentest Notebook*, Last updated 2020. https://refabr1k.gitbook.io/oscp/exploitation-1/ncat-persistent-backdoor

OffSec, “Persistent Netcat Backdoor,” *OffSec.com*, Last updated 2026. https://www.offsec.com/metasploit-unleashed/persistent-netcat-backdoor/

V. Gite, “How to run cron job every minute on Linux/Unix”, *nixCraft*, Last updated Jan. 22, 2022. https://www.cyberciti.biz/faq/how-to-run-cron-job-every-minute-on-linuxunix/

Microsoft, “schtasks create”, *learn.microsoft.com*, Last updated Oct. 10, 2023. https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/schtasks-create

T. Contreras, “From Registry With Love: Malware Registry Abuses”, *splunk.com*, Jan. 19, 2023. https://www.splunk.com/en_us/blog/security/from-registry-with-love-malware-registry-abuses.html

JohnSmith1910, “Microsoft Defender "Suspicious Scheduled Task Launched”, *PSAppDeploymentToolkit*, Feb. 01, 2023 https://discourse.psappdeploytoolkit.com/t/microsoft-defender-suspicious-scheduled-task-launched/4476

Appendices

Appendix A: Defense Configuration

USBGuard

sudo apt install usbguard usbutils udisks2 -> service usbguard status -> sudo -i -> usbguard generate-policy > /etc/usbguard/rules.conf -> exit -> update-rc.d -f usbguard remove -> update-rc.d -s usbguard defaults -> service usbguard start

udev

cd /etc/udev/rules.d -> sudo su -> touch 10-usbblock.rules -> vim 10-usbblock.rules -> in usbblock.rules; #ACTION=="add", ATTR{bInterfaceClass}=="03" RUN+="/bin/sh -c 'echo 0 >/sys$DEVPATH/../authorized'" -> -s -> sed -i 's/#//' /etc/udev/rules.d/10-usbblock.rules; udevadm control --reload-rules -> exit -> exit

Group Policy

In Computer Configuration -> Policies -> Administrative Templates -> System -> Removable Storage Access of GPO Management:

Removable Disks: Deny Execute Access > Enabled

Removable Disks: Deny Write Access > Enabled

Removable Disks: Deny Read Access > Enabled

In Computer Configuration -> Policies -> Administrative Templates -> System -> Device Installation -> Device Installation Restrictions:

Prevent installation of removable devices > Enabled

Appendix B: Quiz Results

11 Participants

Part 1

Question 1: 100% correct

Question 2: 64% correct

Question 3: 91% correct

Question 4: 82% correct

Question 5: 100% correct

Part 1 Average: 87%

Part 2

Question 6: 82% correct

Question 7: 91% correct

Question 8: 82% correct

Question 9: 91% correct

Question 10: 100% correct

Part 2 Average: 89%